

Q. Explain node js modules functions.

Ans.

A module in Node.js is a reusable block of code that can be imported into another file. Each JavaScript file in Node.js is treated as a separate module.

Node.js uses a modular architecture to improve code reusability, maintainability, and security.

1) Core Modules

These are built-in modules provided by Node.js.

Examples:

- http – create web servers
- fs – file system operations
- path – handle file paths
- os – operating system info

Example:

```
const http = require('http');
```

2) Local Modules

User-defined modules created by developers.

Example (math.js):

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = add;
```

3) Third-Party Modules

Installed using **npm (Node Package Manager)**.

Examples:

- express
- lodash
- mongoose

Installation:

```
npm install express
```

Module Exporting

- module.exports – exports a single function or object
- exports – shorthand for exporting multiple items

Advantages of Modules and Functions

- ✓ Code reusability
- ✓ Improved maintainability
- ✓ Better performance
- ✓ Easy debugging
- ✓ Organized application structure

Q. Explain node package manager NPM

Ans.

NPM (Node Package Manager) is the default package management tool for Node.js. It is used to install, manage, share, and update third-party JavaScript libraries (packages) required for Node.js applications.

NPM automatically gets installed along with Node.js.

Purpose of NPM

- Manages project dependencies
- Helps reuse open-source packages
- Simplifies application development
- Maintains package versions

Main Components of NPM

1) NPM Registry

- A public online repository of JavaScript packages
- Contains thousands of open-source libraries
- Example: express, react, lodash

2) package.json File

- A configuration file that stores:
 - Project name and version
 - Dependencies and their versions

Example:

```
{  
  "name": "myapp",  
  "version": "1.0.0",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

Common NPM Commands

Command	Description
npm init	Creates package.json
npm install package-name	Installs a package
npm install -g package-name	Global installation
npm uninstall package-name	Removes a package
npm update	Updates packages
npm list	Shows installed packages

Q. Explain file system in node. Read file, writing file, opening and deleting file and Input/Output operations.

Ans.

Node.js provides a File System (fs) module that allows applications to interact with files and directories on the system. It supports both synchronous and asynchronous file operations.

File System Operations in Node.js

1) Reading a File

Used to read the contents of a file.

Asynchronous Method

```
fs.readFile('data.txt', 'utf8', (err, data) => {  
  if (err) throw err;  
  console.log(data);  
});
```

2) Writing a File

Used to create or write data into a file.

Asynchronous Method

```
fs.writeFile('data.txt', 'Hello Node.js', (err) => {  
  if (err) throw err;  
  console.log("File written successfully");  
});
```

3) Opening a File

Opening a file allows reading or writing using file descriptors.

```
fs.open('data.txt', 'r', (err, fd) => {  
  if (err) throw err;  
  console.log("File opened successfully");  
});
```

4) Deleting a File

Used to remove a file from the system.

```
fs.unlink('data.txt', (err) => {  
  if (err) throw err;  
  console.log("File deleted successfully");  
});
```

Input / Output (I/O) Operations in Node.js

I/O operations refer to reading input and writing output, such as:

- File reading/writing
- Network requests
- Console input/output

The File System module in Node.js provides powerful methods to perform file-related operations such as reading, writing, opening, and deleting files. Its non-blocking I/O model makes Node.js fast and suitable for real-time applications.

Q.Explain Events Event Emitter class, Returning event emitter, Inheriting Events.

Ans.

Node.js follows an event-driven architecture. Many objects in Node.js emit events, and these events are handled using listeners.

The events module provides the EventEmitter class to manage events.

EventEmitter Class

The EventEmitter class is used to create, emit, and handle custom events in Node.js.

```
const EventEmitter = require('events');  
const emitter = new EventEmitter();
```

```
emitter.on('test', () => {  
  console.log("Event triggered");  
});  
emitter.emit('test');
```

Returning Event Emitter

A function can return an EventEmitter object so that the caller can listen for events generated inside the function.

```
function demo() {  
  const emitter = new EventEmitter();  
  setTimeout(() => emitter.emit('done'), 1000);  
  return emitter;  
}
```

Inheriting Events

A class can inherit EventEmitter to create **custom event-driven objects**.

```
class MyClass extends EventEmitter {}
```

The EventEmitter class is the core of Node.js event handling, enabling asynchronous execution through events, returning emitters, and event inheritance.

Node.js follows an **event-driven architecture** where actions are handled using events. The **events module** provides the **EventEmitter** class to create and handle events.

Q. Explain Express.js , web development with Express.js.

Ans.

Express.js is a fast, lightweight, and flexible web framework built on top of Node.js. It is used to develop web applications and RESTful APIs easily.

Features of Express.js

- Simple and minimal framework
- Routing support (GET, POST, PUT, DELETE)
- Middleware support
- Fast server-side development
- Supports REST APIs

Web Development with Express.js

Express.js helps in creating web servers and handling HTTP requests.

```
const express = require('express');
const app = express();
```

```
app.get('/', (req, res) => {
  res.send("Hello Express");
});
```

```
app.listen(3000);
```

Middleware in Express.js

Middleware functions process requests before sending responses, such as logging and authentication.

Express.js simplifies web development by providing routing, middleware, and API support, making it ideal for building scalable web applications.

Q. Explain Functions in Typescript
Ans.

Functions in **TypeScript** are blocks of code used to perform a specific task. TypeScript allows **type-safe functions**, where parameter types and return types can be defined.

Function Declaration

```
function add(a: number, b: number): number {
  return a + b;
}
```

Arrow Function

```
let multiply = (x: number, y: number): number => {
  return x * y;
};
```

Optional and Default Parameters

```
function greet(name: string, msg: string = "Hello"):
void {
  console.log(msg + " " + name);
}
```

Function Overloading

TypeScript supports multiple function signatures.

```
function display(x: string): void;
function display(x: number): void;
function display(x: any): void {
  console.log(x);
}
```

Functions in TypeScript provide type safety, flexibility, and better code readability, making applications more reliable and maintainable.

TypeScript functions help you write clean, bug-free code by catching type errors early.

Q. Explain Classes & interfaces

Ans.

A **class** in TypeScript is a blueprint used to create objects. It supports **object-oriented concepts** such as encapsulation and inheritance.

Example of Class

```
class Student {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  display(): void {
    console.log(this.name);
  }
}
```

Features of Classes:

- Properties and methods
- Constructors
- Access modifiers (public, private, protected)
- Inheritance using extends

Interfaces

An **interface** defines the **structure or contract** that a class must follow. It does not contain implementation.

Example of Interface

```
interface Person {
  name: string;
  display(): void;
}
class User implements Person {
  name: string;
  display(): void {
    console.log(this.name);
  }
}
```

Features of Interfaces:

Classes provide implementation of objects, while interfaces ensure standard structure, making TypeScript programs more organized, reusable, and maintainable.

Difference

- Class → Implementation + structure
- Interface → Only structure

Q. Explain Generics

Ans.

Generics allow creating reusable and type-safe components by using type variables instead of specific data types. They help functions, classes, and interfaces work with different data types while maintaining type checking.

Example:

```
function identity<T>(value: T): T {
  return value;
}
```

Advantages:

- Improves code reusability
- Ensures type safety
- Reduces code duplication

Q. Explain Modules

Ans.

Modules in TypeScript are used to organize code into separate reusable files. Each module has its own scope and can export or import variables, functions, classes, or interfaces.

Example:

```
export function greet() {
  console.log("Hello");
}
import { greet } from './module';
```

Advantages:

- Improves code organization
- Avoids global scope pollution
- Supports reusability and maintainability

Q. Explain Ambients.

Ans.

Ambients in TypeScript are used to declare the shape of existing JavaScript code without providing actual implementation. They help TypeScript understand variables, functions, or libraries that are already defined elsewhere.

Ambient declarations are written using the **declare** keyword.

Example:

```
declare var jquery: any;
```

Advantages:

- Enables use of JavaScript libraries in TypeScript
- Provides type information without changing existing code
- Helps avoid compilation errors

Q. Explain Components - Create, Use, and Manage Components in angular

Ans.

A component is the basic building block of an Angular application. It controls a part of the user interface using HTML (template), CSS (style), and TypeScript (logic).

1) Creating a Component

Components are created using **Angular CLI**.

ng generate component student

A component consists of:

- .ts → Component logic
- .html → View/template
- .css → Styling

Example:

```
@Component({
  selector: 'app-student',
  templateUrl: './student.component.html'
})
export class StudentComponent {}
```

2) Using a Component

A component is used in another template using its **selector**.

<app-student></app-student>

3) Managing Components

Components are managed by:

- Declaring them in **Angular Module**
- Passing data using **@Input()**
- Handling events using **@Output()**
- Using component lifecycle hooks

Example:

@Input() name: string;

Angular components help build structured, reusable, and maintainable user interfaces by separating logic, view, and style.

Q. Explain Directives - add, remove, or manipulate elements in the DOM

Ans.

Directives in Angular are instructions that allow developers to add, remove, or manipulate HTML elements and their behavior in the DOM (Document Object Model).

Types of Directives

1) Structural Directives

Used to add or remove elements from the DOM.

Common structural directives:

- *ngIf
- *ngFor
- *ngSwitch

Example:

<p *ngIf="isLoggedIn">Welcome User</p>

2) Attribute Directives

Used to **change the appearance or behavior** of an element.

Common attribute directives:

- ngClass
- ngStyle

Example:

<p [ngStyle]="{'color':'red'}">Hello</p>

3) Custom Directives

User-defined directives to manipulate DOM elements.

Example:

```
@Directive({
  selector: '[highlight]'
})
export class HighlightDirective {
  constructor(el: ElementRef) {
    el.nativeElement.style.backgroundColor =
    'yellow';
  }
}
```

Angular directives provide powerful control over the DOM by allowing dynamic addition, removal, and modification of HTML elements, making applications interactive and responsive.

Q. Explain Data Binding

Ans.

Data binding is the mechanism that allows synchronization of data between the component (TypeScript) and the view (HTML). It ensures that changes in one are automatically reflected in the other.

1) One-Way Data Binding

Data flows in **one direction**.

a) Interpolation

```
<h2>{{ name }}</h2>
```

b) Property Binding

```
<img [src]="imageUrl">
```

c) Event Binding

```
<button (click)="showMessage()">Click</button>
```

2) Two-Way Data Binding

Data flows **both ways** between component and view using ngModel.

```
<input [(ngModel)]="username">
```

Advantages of Data Binding

- Automatic UI update
- Reduces manual DOM manipulation
- Improves code readability
- Enhances user interaction

Data binding in Angular simplifies communication between the component and the view, making applications dynamic, interactive, and easy to maintain.

Example:

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  name: string = 'Angular Student';
  showMessage() {
    alert('Button Clicked');
  }
}
```

Q. Explain Routing - create and manage routes

Ans.

Routing in Angular is used to **navigate between different views (components)** in a single-page application (SPA) without reloading the page.

Routing lets you navigate betn pages/components in a single-page Angular app without reloading the browser.

Import RouterModule in your module:

```
import { RouterModule, Routes } from
'@angular/router';
```

Define Routes:

```
const routes: Routes = [ { path: '', component:
 HomeComponent }, { path: 'about', component:
 AboutComponent }, ];
```

Register in App Module:

```
@NgModule({ imports:
 [RouterModule.forRoot(routes)], exports:
 [RouterModule] })
```

Route Types:

" → default route

** → wildcard (404)

Nested & parameterized routes are also possible
Angular Routing = Helps build multi-view SPAs
without page reloads.

Example:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from
'@angular/router';
import { HomeComponent } from
'./home.component';
import { AboutComponent } from
'./about.component';
```

```
const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent }
];
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Q. Explain Pipes - Pipes in Angular, use of pipes, Chaining Multiple pipes in angular, Parameterizing a pipe, Filter Pipe, Impure & Pure Pipe, async Pipes.

Ans.

Pipes in Angular are used to transform and format data before displaying it in the view (HTML). Pipes take input data and return transformed output.

Syntax

```
{{ expression | pipeName }}
```

Use of Pipes

Pipes are used to:

- Format strings
- Format dates and currency
- Convert text cases

Built-in Pipes in Angular

Commonly used pipes:

- uppercase
- lowercase
- date
- currency

Example:

```
{{ name | uppercase }}
```

Chaining Multiple Pipes

More than one pipe can be applied to the same expression.

Example

```
{{ name | lowercase | slice:0:4 }}
```

Output of one pipe becomes input of the next pipe.

Parameterizing a Pipe

Some pipes accept **parameters** to control output.

Example

```
{{ today | date:'dd/MM/yyyy' }}
```

```
{{ amount | currency:'INR' }}
```

Filter Pipe

Angular does not provide a built-in filter pipe, but **custom filter pipes** can be created.

Custom Filter Pipe Example

```
@Pipe({ name: 'filterData' })
```

```
export class FilterPipe implements PipeTransform {
  transform(items: any[], value: string): any[] {
    return items.filter(item =>
item.includes(value));
  }
}
```

Pure and Impure Pipes

Pure Pipes

- Default pipe type
- Executes only when input value changes
- Better performance

```
@Pipe({ name: 'myPipe', pure: true })
```

Impure Pipes

- Executes on every change detection
- Used when data changes frequently

```
@Pipe({ name: 'myPipe', pure: false })
```

Async Pipe

The **async pipe** is used to handle **asynchronous data** such as:

- Promises
- Observables

Example

```
{{ data$ | async }}
```

Advantages

- Automatically subscribes and unsubscribes
- Reduces manual subscription code

Advantages of Pipes

- Cleaner templates
- Code reusability
- Improves readability
- Easy data formatting

Pipes in Angular provide an efficient way to transform, filter, format, and display data in views. Features like chaining, parameters, async handling, and pure/impure behavior make pipes powerful for modern Angular applications.

Q. Explain form designing in angular using bootstrap(template driven form)

Ans.

In Angular, template-driven forms are used to design simple forms using HTML templates. Bootstrap is used to make forms responsive and visually attractive using predefined CSS classes.

Requirements

- Angular project
- Bootstrap library
- FormsModule

Steps to Design Form

Step 1: Add Bootstrap

npm install bootstrap

Add in angular.json:

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css"  
]
```

Step 2: Import FormsModule

app.module.ts

```
import { FormsModule } from '@angular/forms';
```

```
@NgModule({  
  imports: [FormsModule]  
})  
export class AppModule { }
```

Example:

```
<form #f="ngForm" class="container">  
  <input type="text" class="form-control" name="name" ngModel placeholder="Name">  
  <input type="email" class="form-control mt-2" name="email" ngModel placeholder="Email">  
  <button class="btn btn-primary mt-2">  
    Submit  
  </button>  
</form>
```

Key Bootstrap Classes Used

- container – layout container
- form-control – input styling
- mb-3 – margin spacing
- btn btn-primary – styled button

Advantages

- Easy to design responsive forms
- Less TypeScript code

Q. Explain Single Page Applications.

Ans.

A Single Page Application (SPA) is a web application that loads only one HTML page and dynamically updates the content without refreshing the entire page. All interactions happen within the same page.

How SPAs Work

- The browser loads a single page initially
- JavaScript dynamically updates content
- Data is fetched using **AJAX / APIs**

Features of SPAs

- Fast navigation
- Smooth user experience
- Uses client-side routing

Advantages

- Improved performance
- Better user experience
- Reduced server load

Disadvantages

- Initial load time may be high
- SEO issues (without SSR)
- More JavaScript dependency

A Single Page Application loads one HTML page and dynamically updates the content without reloading the page.

Benefits of SPAs in Angular:

- Fast & smooth user experience
- No reloads between pages
- Works well with APIs and real-time data

Single Page Applications provide fast, interactive, and smooth web experiences by loading a single page and updating content dynamically using JavaScript.

Q. Explain reactive forms(create,adding validation, grouping, relative forms, async validation)

Ans.

Reactive Forms use TypeScript code to create and manage forms. They give you full control, are scalable, and great for complex forms.

1.Create a Reactive Form

In app.module.ts, import:
`import { ReactiveFormsModule } from '@angular/forms';`

2. Add Validation

```
import { Validators } from '@angular/forms';
form = new FormGroup({
  name: new FormControl("", [Validators.required]),
  email: new FormControl("", [Validators.required,
    Validators.email]) });
```

3. Group Controls

```
form = new FormGroup({
  user: new FormGroup({
    name: new FormControl(""),
    age: new FormControl("")
  })
});
```

4. Reactive Forms (FormBuilder)

```
this.userForm = this.fb.group({
  name: ["", Validators.required],
  email: ["", Validators.email]
});
```

5.Grouping Controls (FormGroup)

```
this.userForm = new FormGroup({
  personal: new FormGroup({
    name: new FormControl(""),
    age: new FormControl("")
  })
});
```

Advantages of Reactive Forms

- Better control & scalability
- Strong validation support
- Easy unit testing

Reactive forms provide a powerful and structured way to create, validate, group, and manage complex forms in Angular, including support for asynchronous validation.

Reactive Forms = More power, more control, ideal for complex or dynamic form logic.

Q. Explain Working with Injectors & Working with Providers.

Ans.

Angular uses **Dependency Injection (DI)** to provide services or objects to components and other classes.

The main parts of DI are:

- **Injector**
- **Provider**
- **Service**

Working with Providers

A provider tells Angular how to create or supply a dependency (service). It defines what object should be injected.

Providing a Service

A provider is configured using the `@Injectable()` decorator or in modules/components.

Example:

```
@Injectable({ providedIn: 'root' })
export class DataService { }
```

Working with Injectors

An **injector** is responsible for **creating instances of services** and **injecting them** where needed.

Angular creates a **hierarchical injector system**:

- Root injector
- Module injector
- Component injector

Injecting a Service

Injector automatically injects services through the **constructor**.

Example:

```
constructor(private dataService: DataService) { }
```

Advantages

- Promotes code reusability
- Loose coupling
- Easy testing
- Better maintainability

Providers define how dependencies are created, while injectors manage and deliver those dependencies to Angular components and services using Dependency Injection.

Q.Explain Reactive form in detail.

Ans.

Reactive Forms are Angular forms where the entire form structure, data model, and validation logic are defined in TypeScript code rather than in HTML. They provide more control, scalability, and testability than template-driven forms.

Create Reactive Form through Code

Reactive forms are created using FormGroup.

```
this.userForm = new FormGroup({
  name: new FormControl(""),
  email: new FormControl("")
});
```

Syncing of HTML and Form

HTML is synchronized with the form model using FormGroup and FormControlName.

```
<form [formGroup]="userForm">
  <input formControlName="name">
  <input formControlName="email">
</form>
```

Adding Validation

Angular provides built-in validators.

```
name: new FormControl("", Validators.required),
email: new FormControl("", Validators.email)
```

Submit Forms

Form submission is handled using ngSubmit.

```
<form [formGroup]="userForm"
(ngSubmit)="onSubmit()">
onSubmit() {
  console.log(this.userForm.value);
}
```

Grouping (FormGroup)

Related fields are grouped using nested FormGroup.

```
this.userForm = new FormGroup({
  personal: new FormGroup({
    name: new FormControl(""),
    age: new FormControl("")
  })
});
```

Form Control Arrays

FormArray is used when multiple similar inputs are required.

```
skills = new FormArray([
  new FormControl('Angular'),
  new FormControl('Node')
]);
this.userForm.addControl('skills', this.skills);
```

Custom Validators

Custom validation logic defined by developer.

```
function forbiddenName(control: AbstractControl)
{
  return control.value === 'admin' ? { forbidden:
true } : null;
}
```

Usage:

```
name: new FormControl("", forbiddenName)
```

Reactive Forms (FormBuilder)

FormBuilder simplifies form creation.

```
this.userForm = this.fb.group({
  name: ['', Validators.required],
  email: ['', Validators.email]
});
```

Async Validation

Used when validation requires server response.

```
email: new FormControl("", null, this.checkEmail)
checkEmail(control: AbstractControl) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(control.value === 'test@gmail.com' ? {
invalid: true } : null);
    }, 1000);
  });
}
```

Value Changes and Reacting to Status

Reactive forms emit events when form values or status change.

```
this.userForm.valueChanges.subscribe(value => {
  console.log(value);
});
```

```
this.userForm.statusChanges.subscribe(status => {
  console.log(status);
});
```

Q. Explain ASYNCHRONOUS OPERATIONS & HTTP in detail.

Ans.

Asynchronous operations allow tasks like HTTP requests, file loading, and timers to execute without blocking the main program flow. Angular supports async operations to build responsive and high-performance applications.

Promises

A **Promise** represents a value that may be available **now, later, or never**.

States of Promise

- Pending
- Fulfilled
- Rejected

Example:

```
let p = new Promise((resolve, reject) => {
  resolve("Success");
});
p.then(result => console.log(result));
```

Working with Observables

Observables are used extensively in Angular for handling **streams of asynchronous data**. They can emit **multiple values over time**.

Example:

```
this.http.get(url).subscribe(data => {
  console.log(data);
});
```

EventEmitter

EventEmitter is used for **component-to-component communication**, especially between **child and parent components**.

Example:

```
@Output() notify = new EventEmitter<string>();
this.notify.emit("Hello");
```

Async Pipes

The **async pipe** is used in templates to handle **observables or promises automatically**.

Example:

```
{{ data$ | async }}
```

Benefits

- Auto subscribe/unsubscribe
- Cleaner code

Handling HTTP Request / Response

Angular provides **HttpClient** to perform HTTP operations.

Common HTTP Methods

- GET – fetch data
- POST – send data
- PUT – update data
- DELETE – remove data

Example:

```
this.http.get('api/users')
```

Headers & Request Settings

HTTP headers are used to send **metadata** such as content type and authorization.

Example:

```
let headers = new HttpHeaders({
  'Content-Type': 'application/json'
});
```

```
this.http.post(url, data, { headers });
```

Providing HTTP

To use HTTP services, **HttpClientModule** must be imported.

```
import { HttpClientModule } from
'@angular/common/http';
```

```
@NgModule({
  imports: [HttpClientModule]
})
export class AppModule { }
```

Angular provides powerful asynchronous features such as Promises, Observables, EventEmitter, Async Pipes, and HttpClient to handle non-blocking operations efficiently, enabling scalable and responsive web applications.

**Q. Explain Next.js Pages(Static and Dynamic)
Ans.**

In Next.js, a page is a React component stored inside the pages directory. Each file automatically becomes a route based on its file name. Next.js supports both static pages and dynamic pages.

1.Static Pages

Static pages are pages whose content is generated at build time. They are fast and SEO-friendly.

Example

pages/about.js

```
export default function About() {  
  return <h1>About Page</h1>;  
}
```

Accessed using:

/about

Features

- Fixed routes
- High performance
- Content does not change dynamically

Dynamic Pages

Dynamic pages are used when URLs change dynamically, such as user profiles, product pages, etc. File names use square brackets [].

Example

pages/user/[id].js

```
import { useRouter } from 'next/router';  
export default function User() {  
  const router = useRouter();  
  return <h1>User ID: {router.query.id}</h1>;  
}
```

Accessed using:

/user/101

/user/abc

Advantages of Next.js Pages

- File-based routing
- No manual route configuration
- SEO friendly
- Supports static & dynamic content

Next.js pages simplify routing by using the file system. Static pages are best for fixed content, while dynamic pages are ideal for data-driven and user-specific routes.

**Q. Style next js app with css module
Ans.**

CSS Modules in Next.js provide **locally scoped CSS** to components. This avoids class-name conflicts and makes styles easy to maintain. Next.js supports CSS Modules **by default**, without any extra configuration.

A **CSS Module** is a CSS file named with module.css. The styles defined inside it are **automatically scoped** to the component that imports them.

Steps to Style Next.js App with CSS Modules

1) Create a CSS Module File

```
/* styles/Home.module.css */  
.title {  
  color: blue;  
  font-size: 24px;  
}
```

2) Import the CSS Module in Page

import styles from '../styles/Home.module.css';

3) Apply Styles using className

```
export default function Home() {  
  return <h1 className={styles.title}>Welcome to  
Next.js</h1>;  
}
```

How CSS Modules Work

- Class names are **hashed automatically**
- Styles apply only to the importing component
- Prevents global CSS conflicts

Advantages of CSS Modules

- Scoped & reusable styles
- No naming conflicts
- Easy to maintain
- Built-in support in Next.js

CSS Modules provide a clean and modular way to style Next.js applications. They ensure scoped styling, better code organization, and improved maintainability.

Q. Explain Create a Next.js App(setup, edit, navigate, deploy, pre-render, assets)

Ans.

1.Setup

A Next.js application is created using **create-next-app**, which sets up all required configuration automatically.

```
npx create-next-app myapp
```

```
cd myapp
```

```
npm run dev
```

→ Runs the app on <http://localhost:3000>

2. Editing the Page

Pages are React components stored inside the pages folder.

Editing pages/index.js updates the home page.

```
export default function Home() {  
  return <h1>Welcome to Next.js</h1>;  
}
```

3.Navigate Between Pages

Next.js provides file-based routing and navigation using the Link component.

```
import Link from 'next/link';
```

```
<Link href="/about">About</Link>
```

Each file in the pages directory becomes a route automatically.

4. Assets, Metadata, and CSS

- Assets (images, icons) are stored in the public folder.
- Metadata is handled using the Head component.
- CSS can be global or modular.

```
import Head from 'next/head';
```

```
<Head>
```

```
  <title>My App</title>
```

```
</Head>
```

CSS Modules use .module.css files.

5. Pre-rendering and Data Fetching

Next.js pre-renders pages for better performance and SEO.

Data Fetching Methods

- `getStaticProps()` – Static Generation

- `getServerSideProps()` – Server-Side Rendering

```
export async function getStaticProps() {  
  return { props: { data: [] } };  
}
```

6. Dynamic Routes

Dynamic routes are created using **square brackets**.

pages/user/[id].js

Access URL:

/user/101

Dynamic routing is useful for user profiles and product pages.

7. API Routes

Next.js allows creating backend APIs inside the pages/api folder.

```
export default function handler(req, res) {  
  res.status(200).json({ message: "Hello API" });  
}
```

Accessed via /api/hello

8. Deploying Next.js App

Next.js apps are commonly deployed on Vercel.

Steps:

1. Build app
2. npm run build
3. Deploy using Vercel
4. vercel

Other platforms include Netlify and AWS.

Next.js simplifies web development by offering built-in routing, styling, APIs, pre-rendering, and easy deployment, making it ideal for fast and SEO-friendly web applications.

